

ویژگی های الگوریتم های مرتب سازی

الگوریتم	پایداری	درجا / غیردرجا	بهترین حالت	حالت متوسط	بدترین حالت
Bubble Sort	پایدار	درجا	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	پایدار	درجا	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	ناپایدار	درجا	$O(n^2)$	$O(n^2)$	$O(n^2)$
Quick Sort	ناپایدار	درجا	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge Sort	پایدار	غیردرجا	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Counting Sort	پایدار	غیردرجا	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix Sort	پایدار	غیردرجا	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$
Heap Sort	ناپایدار	درجا	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

مزایا و معایب الگوریتم های مرتب سازی

کاربرد مناسب	نقاط ضعف	نقاط قوت	الگوریتم
داده های کوچک، اهداف آموزشی	برای داده های بزرگ بسیار کند است	ساده و قابل درک، در آرایه تقریباً مرتب سریع است	Bubble Sort
آرایه های کوچک، داده های تقریباً مرتب	برای داده های بزرگ و نامرتب کند است	ساده، سازگار با داده های تقریباً مرتب، سربار کم	Insertion Sort
کوچک n وقتی هزینه جابجایی زیاد و است	سازگار $O(n^2)$ ، تعداد مقایسه همیشه نیست	$O(n)$ حداکثر (تعداد جابجایی کم)	Selection Sort
مرتب سازی عمومی آرایه ها	ناپایدار است، $O(n^2)$ بدترین حالت	در عمل بسیار سریع، سازگار با حافظه کش حافظه کم	Quick Sort
نیاز به پایداری، مرتب سازی خارجی و لیست پیوندی	نیاز به حافظه اضافی، کمی کندتر از Quick Sort	پایدار $O(n \log n)$ کارایی تضمینی	Merge Sort
اعداد صحیح با دامنه کوچک	برای دامنه بزرگ حافظه و زمان زیاد می شود	بسیار سریع وقتی دامنه مقادیر کوچک است	Counting Sort
حجم زیاد اعداد صحیح یا رشته با طول ثابت	محدود به داده های خاص، نیاز به حافظه اضافی	برای اعداد یا رشته با طول ثابت بسیار سریع است	Radix Sort
حافظه محدود همراه با نیاز به تضمین کارایی	کارایی کش ضعیف، ناپایدار	بدون حافظه $O(n \log n)$ درجا، تضمین اضافی	Heap Sort

مراحل مرتب سازی روی آرایه ورودی

آرایه ورودی: [25, 17, 8, 42, 17, 3, 19, 8, 33]

Bubble Sort (الف)

آرایه اولیه: [25, 17, 8, 42, 17, 3, 19, 8, 33]

پس از پاس ۱: [17, 8, 25, 17, 3, 19, 8, 33, 42]

پس از پاس ۲: [8, 17, 17, 3, 19, 8, 25, 33, 42]

پس از پاس ۳: [8, 17, 3, 17, 8, 19, 25, 33, 42]

Abbas Ashoury

پس از پاس ۴: [8, 3, 17, 8, 17, 19, 25, 33, 42]

پس از پاس ۵: [3, 8, 8, 17, 17, 19, 25, 33, 42]

پس از پاس ۶ (بدون تغییر، مرتب شده): [3, 8, 8, 17, 17, 19, 25, 33, 42]

آرایه نهایی: [3, 8, 8, 17, 17, 19, 25, 33, 42]

ب) Insertion Sort

درج عنصر ۱۷: [17, 25, 8, 42, 17, 3, 19, 8, 33]

درج عنصر ۸: [8, 17, 25, 42, 17, 3, 19, 8, 33]

درج عنصر ۴۲ (بدون تغییر): [8, 17, 25, 42, 17, 3, 19, 8, 33]

درج عنصر ۱۷: [8, 17, 17, 25, 42, 3, 19, 8, 33]

درج عنصر ۳: [3, 8, 17, 17, 25, 42, 19, 8, 33]

درج عنصر ۱۹: [3, 8, 17, 17, 19, 25, 42, 8, 33]

درج عنصر ۸: [3, 8, 8, 17, 17, 19, 25, 42, 33]

درج عنصر ۳۳: [3, 8, 8, 17, 17, 19, 25, 33, 42]

آرایه نهایی: [3, 8, 8, 17, 17, 19, 25, 33, 42]

ج) Merge Sort

تقسیم آرایه اصلی به دو نیمه:

چپ [25, 17, 8, 42]

راست: [17, 3, 19, 8, 33]

تقسیم نیمه چپ:

[25, 17] و [8, 42]

تقسیم بیشتر و مرتب سازی:

[17, 25] به [25, 17]

Abbas Ashoury

[8, 42] به [8, 42]

[8, 17, 25, 42]: ادغام نیمه چپ

تقسیم نیمه راست:

[17, 3] و [19, 8, 33]

[3, 17]: [17, 3] مرتب سازی

[19, 8, 33]: تقسیم

[8, 33] و [19]

[8, 33]: [8, 33] مرتب سازی

[8, 19, 33]: ادغام [۱۹] با [8, 33]

[3, 8, 17, 19, 33]: ادغام نیمه راست

[3, 8, 17, 19, 33] و [8, 17, 25, 42]: ادغام نهایی (نیمه چپ و نیمه راست)

[3, 8, 8, 17, 17, 19, 25, 33, 42]: آرایه نهایی

د) Counting Sort

گام ۱: یافتن کمترین و بیشترین مقدار

کمترین مقدار = ۳

بیشترین مقدار = ۴۲

گام ۲: شمارش تکرار هر مقدار

مقدار ۳: تعداد ۱

مقدار ۸: تعداد ۲

مقدار ۱۷: تعداد ۲

مقدار ۱۹: تعداد ۱

مقدار ۲۵: تعداد ۱

مقدار ۳۳: تعداد ۱

مقدار ۴۲: تعداد ۱

سایر مقادیر: تعداد ۰

گام ۳: محاسبه جمع تجمعی (تعداد عناصر کوچک تر یا مساوی هر مقدار)

تا ۳: ۱

تا ۸: ۳

تا ۱۷: ۵

تا ۱۹: ۶

تا ۲۵: ۷

تا ۳۳: ۸

تا ۴۲: ۹

گام ۴: قرار دادن عناصر در آرایه خروجی بر اساس جمع تجمعی (برای حفظ پایداری از انتهای آرایه ورودی شروع می شود)

[3, 8, 8, 17, 17, 19, 25, 33, 42]: آرایه نهایی